

Analysis: Erdős–Szekeres

Get the information

The real trick to this problem is squeezing as much information as possible from the sequences $A[i]$ and $B[i]$. We will get information out in the form of inequalities between various elements of the sequence X .

First notice that if we have two indices $i < j$, and $A[i] \geq A[j]$, then $X[i] > X[j]$. Indeed, if it were otherwise, then the increasing sequence of length $A[i]$ with $X[i]$ as its largest element could be extended by adding $X[j]$ at the end to obtain an increasing sequence with $X[j]$ at the end, and we would have $A[j] \geq A[i] + 1$. This allows us to add some inequalities X has to satisfy. We can add symmetric ones for B — if $i < j$ and $B[i] \leq B[j]$, then $X[i] < X[j]$.

These inequalities are not enough, however, to reconstruct X . Indeed, if we take $A[i] = i + 1$ and $B[i] = N - i$, we get no inequalities to consider, but at the same time not all permutations X are going to work. The problem is that while we know what we need to do with X so that no longer subsequences exist, we still need to guarantee that long enough sequences exist.

This is relatively simple to accomplish, though. If $A[i] > 1$, then the increasing subsequence ending at $X[i]$ is formed by the extension of some sequence of length $A[i] - 1$. This means that $X[i]$ has to be larger than $X[j]$ for some $j < i$ with $A[j] = A[i] - 1$. The previous set of inequalities guarantees that of the set of all such j (that is, j which are smaller than i and have $A[j] = A[i] - 1$) the one with the smallest $X[j]$ is the largest j . Thus, it is enough to find the largest j with $j < i$ and $A[j] = A[i] - 1$ and add $X[j] < X[i]$ to our set of inequalities. We again do the symmetric thing for B .

Use the information

Notice that the inequalities we have are indeed enough to guarantee that any sequence X satisfying them will lead to the A and B we want. It's relatively easy to check the first set of inequalities guarantees the numbers A and B will not be larger than we want, while the second set of inequalities guarantee they will be large enough.

We now reduced the problem to finding the lexicographically smaller permutation satisfying a given set of inequalities. To find the lexicographically smallest result we are foremost interested in minimizing the first value in X . To this end, we will simply traverse all the inequalities that the first value in X has to satisfy (that is, iterate over all the elements we know to be smaller, then all elements we know to be smaller than these elements, etc.). We can do this, e.g., through a DFS. After learning how many elements of X have to be smaller than $X[1]$, we can assign the smallest possible value (this number + 1) to $X[1]$. Now we need to assign numbers smaller than $X[1]$ to all these elements, in the lexicographically smallest way.

Note that how we do this assignment will not affect the other elements (since they are all going to be larger than $X[1]$, and so also larger than everything we assign right now). Thus, we can assign this set of elements so that it is lexicographically smallest. This means taking the earliest of these elements, and repeating the same procedure recursively (find all elements smaller than it, assign the appropriate value, recurse). Note that once some values have been assigned, we need to take that into account when assigning new values (so, if we already assigned 1, 3 and 10; and now we know that an element we're looking at is larger than 4 others, the smallest value we can assign to it is 7).

Such a recursive procedure will allow us to solve the problem. For each element, we are going to traverse the graph of inequalities to find all the elements smaller than it is ($O(\mathbf{M})$ time if we do a DFS, where $\mathbf{M}$ is the number of inequalities we have), then see what's the smallest value we can assign ($O(\mathbf{N})$ with a linear search, we can also go down to $O(\log \mathbf{N})$, but don't need to), and recurse. This gives us, pessimistically, $O(\mathbf{N}^3)$ time — easily good enough for the small testcase, but risky for the large.

Compress the information

Following the exact procedure above we can end up with $O(\mathbf{N}^2)$ inequalities. This will be too much for us (at least for the large dataset), so we will compress a bit.

The problem is with inequalities of the first type — for indices smaller than a given i there can be many with A larger or equal to $A[i]$. A trick we can use, though, is to take only one inequality — find the largest $j < i$ with $A[j] = A[i]$, and insert only the inequality $X[j] \geq X[i]$.

Any other $k < j$ with $A[k] = A[i]$ will follow from transitivity — there will be a sequence of indices with A equal to $A[i]$ connecting k to i . Any k with $A[k] > A[i]$ will also follow, since $X[k]$ will have to be greater than some $X[l]$ for $A[l] = A[i]$ and $l < k$ (and thus $< i$). This means we can reduce our set of inequalities down to a set of $O(\mathbf{N})$, which means that each DFS traversal step will take only $O(\mathbf{N})$ time, and the solution to the whole problem will run in $O(\mathbf{N}^2)$.

Interesting fact: It is also possible to solve this problem in $O(\mathbf{N} \log \mathbf{N})$. Can you figure it out?

Analysis: Many Prizes

Let's begin by an observation that will make our life a bit easier: if we reverse all the team numbers (that is, team 0 becomes team 2^N-1 , team 1 becomes team 2^N-2 , etc.), without changing the tournament list. This will result in the final ranks of the teams also being reversed, since it's relatively easy to see that all the records will be reversed (wins becoming losses and vice versa). This means that the problem of having a team rank as low as possible to get into the first P is equivalent to the problem of a team ranked as high as possible to get into the bottom P (or, in other words, not get into the top $2^N - P$). Thus, if we are able to answer the question what is the lowest rank that can possibly get into the top P , we can run the same code to see what's the lowest rank that can possibly be in the top $2^N - P$, subtract one, and reverse, and this will be the lowest rank that will always get a prize. This means we only have to solve one problem — lowest ranked team guaranteed to be in the top P .

For the small dataset we have at most 1024 teams, which means we can try to figure out the tournament tree that will give a given team the best position, do this for all teams, and see which team was the lowest-ranked one to get a prize. Let's skip this, however, and jump straight to the solution for the large dataset, where 2^{50} teams clearly disallow any direct approaches.

The key observation we can make here is that if we have a team we want to be as high as possible, we can do it with a record that includes a string of wins followed by a string of losses, and nothing else. This sounds surprising, but is actually true. Imagine, to the contrary, that the team (let's call them **A**) has a loss (against some team **B**) followed by a win against **C**, who played **D** in the previous round. Note that up to the round where **A** plays **B** the records of the four teams were identical. Also note that all the tournament trees of the four teams so far are disjoint, and so we can swap them around. In particular, we can swap team **C** and all its tree with team **B** and all its tree. Since we swap whole trees, the records of teams don't change, so now team **A** will play **C** in the first match and win — and so its record is going to be strictly better than it was, no matter what happens next. Thus, any ordering in which team **A** has a loss followed by a win is suboptimal.

This allows us to solve the problem of getting the highest possible rank for a given team. We simply need to greedily win as much as we can. If we're the worst team, we can't win anything. If we're not, we can certainly win the first match. The second match will be played against the winner of some match, so in order to win it we need to be better than three teams. To win two matches, we need to be better than seven teams, and so on.

We can also reverse the reasoning to get the lowest-ranked team that can win the prize. First, let's ask how many matches does one need to win in order to win a prize. If you win no matches, you are in the top 2^N (not a huge achievement!). If you win one, you are in the top $2^N - 1$. And so on. Once we know how many matches you need to win, you directly know how many teams you need to be better than. Simple python code follows:

```
def LowRankCanWin(N, P):
    if P == 0:
        return -1
    matches_won = 0
    size_of_group = 2 ** N
    while size_of_group > P:
        matches_won += 1
        size_of_group /= 2
```

```
    return 2 ** N - 2 ** matches_won

def ManyPrizes(N, P):
    print 2 ** N - LowRankCanWin(N, 2 ** N - P) - 2, LowRankCanWin(N, P)
```

Analysis: Multiplayer Pong

Preliminaries

We hope you took our Fair Warning, which we fairly repeated in Fair and Square this year — we consider Big Integers to be fair game, so if your language doesn't natively support them, you'd better have a library to handle them ready.

This problem seems to be about fractions initially, since the ball can hit the walls at fractional positions. There's a way to move it to integers (man, Big Fractions would be too much). The way to go about it is to scale things. Let's scale the time, so now there are V_X units to a second, and scale vertical distances, so that there are V_X units to the old unit. This means the vertical speeds stay the same, horizontal speeds get V_X times smaller, horizontal distances stay the same, and vertical distances grow V_X times larger. In implementation terms, this means we shrink V_X to 1 and grow A times V_X — and now suddenly the ball moves a integral number of units upwards and one unit to the side in each unit of time (and so will hit the vertical walls in integral moments of time).

The above assumes V_X is positive. If it's zero, then the ball will never hit the walls, and so the game ends in a draw. If it's negative, we can flip the whole board across the Y axis and swap the teams to make V_X change signs. Similarly, we can assume V_Y positive — if it's zero, putting all paddles in the single impact point will guarantee a draw, and if it's negative, we can flip the board vertically.

So now the ball will hit a given vertical wall every $2B$ units of time. It's also relatively easy to figure out what is the position of the impact. If there were no vertical walls, the ball would hit at the initial hit position ($Y + (B - X) V_Y$), and then at $2BV_Y$ intervals from there. To calculate the positions in real life, notice that every $2A$ upwards the ball is in the same position again (so we can take impact points modulo $2A$), and if that number is larger than A , the ball goes in the other direction, and the impact position is $2A$ minus whatever we calculated.

Many bounces

So now we know how to calculate hit positions, so we can just simulate and see who loses first, right? Well, wrong, because the ball can bounce very many times. Even in the small dataset, the number of bounces can go up to 10^{11} , too much for simulation.

Notice that the positions of the paddles are pretty much predetermined. The paddle of a given player has to be exactly at the point of impact when it is the player's turn to bounce it, and then the only question is whether the player will have enough time to reach the next point of impact before the ball. With N paddles on a team, and V being the speed of the paddle, the player can move the paddle by $2BV_N$ before the next impact. The ball, in the same time, will move by $2BV_YN$, but while the paddle moves in absolute numbers, the ball moves "modulo $2A$ with wrapping", as described above.

If the distance the ball moves (modulo A) is smaller or equal to the distance the paddle moves, the paddle will always be there in time. The interesting case is when it is not so. In this case, there is still a chance for the paddle to be in the right place on time due to the "wrapping effect". For instance, if the ball moves by $A+1$ with each bounce, and the first bounce happens at $A/2$, the next one will happen at $A/2 - 1$, the next one at $A/2 + 2$, and so on — so the initial

bounces will be pretty close by. We can calculate exactly the set of positions where the ball hitting the wall will allow the paddle to catch up, and it turns out that it is two intervals modulo $2A$.

So now the question becomes "how long can the ball bounce without hitting a prohibited set of two intervals", which is easy to reduce to "without hitting the given interval". This is a pure number-theoretic question: we have an arithmetic sequence $I + KS$, modulo $2A$, and we are interested in the first element of this sequence to fall into a given interval.

Euclid strikes again

There is a number of approaches one can take to solving this problem. We will take an approach similar to the Euclidean algorithm. First, we can shift everything modulo $2A$ so that I is zero, and we are dealing with the sequence KS . Also, we can shift the problem so that $S \leq A$ — if not, we can (once again) flip the problem vertically. Thus, the ball will bounce at least twice before wrapping around the edge of $2A$.

We can calculate when is the first time the ball will pass the beginning of the forbidden interval (by integral division). If at this point the ball hits the forbidden interval, we are done. Otherwise, the ball will travel all the way to $2A$, and then wrap around and hit the wall again at some position P smaller than S . Notice that in this case the interval obviously is shorter than S .

Now, the crucial question is what P is. It's relatively easy to calculate for what values of P will the next iteration land in the interval (if the interval is $[kS + a, kS + b]$ for some k, a, b , then the interesting set of values of P is $[a, b]$). If P happens to be in this interval, we can again calculate the answer fast. If not, however, we will do another cycle, and then hit the wall (after cycling) at the point $2P$, mod S . Notice that this is very similar to the original problem! We operate modulo S , we increment by P with each iteration, and we are interested in when we enter the interval $[a, b]$.

Thus, we can apply recursion to learn after how many cycles will we finally be in a position to fall into the original interval we were interested in. So we make that many full cycles, and then we finish with a part of the last cycle to finally hit the interval.

Complexity analysis

All the numbers we will be operating on will have at most $D = 200$ digits in the large dataset, so operations on them will take at most $O(D^2 \log D)$ time (assuming a quadratic multiplication implementation and a binary-search implementation of division). Each recursion step involves a constant number of arithmetic operations plus a recursive call, for which the number A (the modulo) decreases at least twice. This means we make at most $O(D)$ recursion steps — so the whole algorithm will run in $O(D^3 \log D)$ time, fast enough allowing even some room for inefficiency.

Analysis: Ticket Swapping

The small dataset

Note that in this problem we treat the passengers as one "player" in a game, and assume they all cooperate to pay as little as possible in total to the city. This means it doesn't matter who actually pays for a given entry card. In particular, when the train leaves a station, the charge on each entry card in the train increases (by $N - i$, where i is the number of stations this card traveled so far). Since all passengers want to exit the subway eventually, all entry cards will have to be paid — so we might just as well immediately subtract this cash from the passenger "total" and move on.

As long as nobody exits the train, there's no need to exchange entry cards. Only once someone needs to exit, the passengers (along with the ones who just entered) need to gather and figure out which entry cards do the passengers who are just leaving take with them. They should choose the entry cards that have been on the train for the shortest amount of time so far, since on each subsequent stop the price for such an entry card will be larger than the price for any card that has been on the train longer (as the price is $N - i$, where i is the number of stations the card traveled so far). This means that at every station, the passengers should pool all the cards together, and then whoever wants to exit takes the entry cards with the smallest distance.

For the small dataset, a naive implementation of this algorithm will work. We can process station by station, holding a priority queue (or even just any collection) of the entry cards present. When anyone wants to exit, we iterate over the collection to find the card that has been on the train for the shortest amount, add its cost to the total passengers cost and remove it from the set. We also need to figure out how much the passengers *should* pay, but fortunately that's easy.

The large dataset

The large data set needs a bit more subtlety. With the insane amounts of passengers and stations we will need to avoid processing unnecessary events. First of all, we should process only the stations at which someone wants to enter or exit. This will make us process only $O(M)$ stations, much less than the N we would process otherwise. Moreover, we should avoid processing passengers one by one.

To this end, notice that the order in which we want to give exit cards to passengers is actually a LIFO stack — whichever card came in last will go out first. So, we can keep the information about entry cards present in the train in a [stack](#). Whenever a new group of passengers comes in, we take their entry cards and put them onto the stack (as one entry, storing the number of cards and the entry station). Whenever any group wants to leave, we go through the stack. If the topmost group of cards is big enough, we simply decrease its size, pay for what we took, and continue. If not, we take the whole group of cards, pay for it, decrease the amount of cards we need by the size of the group, and proceed through the stack.

This algorithm will take only $O(M)$ time in total to process all the passengers — we will put on the stack at most M times, so we will take a whole group from the stack at most M times, and each group of passengers will decrease a group size (not take the whole group of cards) at most once, so in total — at most M such operations in the whole algorithm. Additionally, we need to sort all the events (a group of passengers entering or leaving) up front, so we are able to solve the whole problem in $O(M \log M)$.

Finally, when implementing, one needs to be careful. Due to the large amounts of stations and passengers involved, we have to use modular arithmetic carefully, because — as always with modular arithmetic — we risk overflow. In particular, whenever we multiply three numbers (which we do when calculating how much to pay for a group of tickets), we need to take care to apply the modulo after multiplying the first two.